

■ Duolingo Clone

Android Nativo · Kotlin · RecyclerView + DiffUtil

Taller 6 — Native UI Re-Engineering & UX Analysis

Evelin Ximena Rocha Rocha

Aplicaciones Móviles · 2026

Fase A: Análisis UX

Fase B: Implementación

Fase C: Mejora

¿Qué es este proyecto?

Reverse-engineering de la app Duolingo, reconstruida desde cero en **Android nativo con Kotlin**, aplicando buenas prácticas de arquitectura, animaciones y UX.

- Analizar patrones de UX de Duolingo
- ■ Diseñar arquitectura con Fragments + RecyclerView
- Implementar micro-animaciones nativas
- Proponer una mejora propia (Fase C)

- Pantalla Home — Ruta de lecciones
- Pantalla Liga — Tabla de posiciones
- Pantalla Perfil — Logros desbloqueados
- Navegación con Bottom Navigation Bar

FAS
E A

Análisis UX de Duolingo

Patrón observado	Implementación en el clon
Ruta de lecciones con conectores verticales	RecyclerView + item_lesson.xml con viewConnector
Estados visuales: completado / activo / bloqueado	applyCompletedState / applyActiveState / applyLockedState
Tabla de clasificación con avatar de color	LeaderboardAdapter con initials + color de avatar
Logros con barra de progreso individual	AchievementAdapter con progressAchievement visible/gone
Navegación por pestañas inferior	BottomNavigationView con 3 ítems
Micro-animación al tocar lección	ObjectAnimator (scale) con rebote press/release

■ Estructura de paquetes

- `com.example.duolingoclone`
- data/model/ → Lesson, LeaderboardUser, Achievement
- data/ → MockDataRepository (object singleton)
- ui/
- home/ → HomeFragment + LessonAdapter
- league/ → LeagueFragment + LeaderboardAdapter
- profile/ → ProfileFragment + AchievementAdapter

■ Tecnologías clave

- **ViewBinding** — acceso tipado a vistas, sin `findViewById`
- **ListAdapter + DiffUtil** — diff eficiente de listas
- **ObjectAnimator** — animaciones sin librerías externas
- **Fragment + FragmentManager** — navegación liviana
- **BottomNavigationView** — navegación principal
- **DividerItemDecoration** — separadores en la lista

Las 3 Pantallas Principales

■ HOME

- RecyclerView con 20 lecciones
- Conectores verticales entre ítems
- Estado: completado (verde) / activo / bloqueado
- Pulso suave en lección activa actual
- Barra de meta diaria (65/100 XP)

■ LIGA

- 15 usuarios en tabla de posiciones
- Avatar con iniciales y color único
- Usuario propio resaltado en morado
- DividerItemDecoration entre filas
- Datos: nombre + XP + posición

■ PERFIL

- 12 logros en grid/lista
- Estado: desbloqueado / bloqueado
- Barra de progreso por logro
- Emoji representativo por logro
- Descripción de condición de desbloqueo

Micro-animaciones Nativas

Animación	Código (Kotlin)	Efecto UX
Pulso en lección activa	<code>ObjectAnimator scaleX/Y 1f→1.04f→1f repeatCount=INFINITE, duration=1400ms</code>	Guía visual: 'aquí debes continuar'
Press / tap feedback	<code>AnimatorSet: down (scale→0.93f, 90ms) + up (scale→1f, 90ms) playSequentially</code>	Respuesta táctil inmediata al toque
Transición entre pantallas	<code>setCustomAnimations(fade_in, fade_out)</code>	Cambio fluido sin saltos bruscos
Estado de logro desbloqueado	<code>alpha = 0.35f (bloqueado) alpha = 1f (disponible)</code>	Feedback visual de progreso claro

■ Barra de Meta Diaria Siempre Visible

Problema identificado en Duolingo original:

El progreso XP diario solo se muestra dentro de una pantalla específica, lo que obliga al usuario a navegar para saber cuánto le falta para su meta.

Solución implementada:

Se agregó una **ProgressBar horizontal + texto «65/100 XP»** fija en el header de HomeFragment, siempre visible sin importar la posición del scroll.

Código clave:

```
binding.progressBarDailyGoal.max = total
binding.progressBarDailyGoal.progress = current
binding.tvDailyGoalProgress.text = "$current / $total XP"
```

Stack Técnico & Conclusiones

Kotlin

Android SDK

ViewBinding

RecyclerView

DiffUtil

ObjectAnimator

- RecyclerView con ListAdapter + DiffUtil evita re-renders innecesarios
- ViewBinding elimina NullPointerException en vistas
- Arquitectura Fragment modular y escalable
- Animaciones 100% nativas — sin dependencias externas
- MockDataRepository centraliza todos los datos de prueba

- Aprendí a clonar UI complejas analizando patrones reales
- Comprendí el valor de DiffUtil para listas de aprendizaje
- Las micro-animaciones mejoran significativamente el UX
- Un buen análisis (Fase A) guía toda la implementación
- La mejora propia refuerza pensamiento crítico de producto